SUPPLY CHAIN MANAGEMENT SYSTEM WITH ANALYTICS

AAT Mini Project Report

Technology: MySQL

Title: Supply Chain Management System with Analytics

Student Name: PRANAV BALAJI. P

Reg.no : 711525BCS118

Department / Class: BE. CSE / C

Subject code : B25CST302

Subject : DataBase Management System

College: Kalaignarkarunanidhi Institute of Technology

1. Abstract

The Supply Chain Management System with Analytics is a database project designed to store and manage information about suppliers, products, customers, orders, and deliveries. SQL queries are used to generate useful reports such as total sales, sales by order, sales by product category, low-stock products, and delivery status. The project demonstrates relational database design, primary and foreign keys, joins, grouping, and aggregate functions.

2. Objectives

- Maintain supplier, product, customer, order, and delivery records.
- Track product prices and available stock.
- Calculate order totals and overall sales.
- Identify products with low inventory.
- Find high-revenue products and compare category sales.
- Monitor delivery progress using delivery-status reports.

3. Software Requirements

- Database: MySQL 8.x (or a compatible MySQL version)
- Interface: MySQL Workbench, phpMyAdmin, or MySQL command line
- Operating system: Windows, Linux, or macOS
- Input: SQL statements and sample business records
- Output: Tables and analytical reports

4. Database Design

The database is named SupplyChainDB. It contains six tables: Suppliers, Products, Customers, Orders, OrderDetails, and Deliveries.

Table	Purpose	Primary key
Suppliers	Supplier name and city	supplier_id
Products	Product category, price, stock and supplier	product_id
Customers	Customer name and city	customer_id
Orders	Customer order date and status	order_id

OrderDetails	Products and quantities in each order	(order_id, product_id)
Deliveries	Delivery date and delivery status	delivery_id

Relationship summary

- One supplier can supply many products.
- One customer can place many orders.
- One order can contain multiple products through OrderDetails.
- Each delivery record is linked to an order.

5. Complete SQL Program

Run the following sections in order. The sample records are fictional and are included to demonstrate the project.

5.1 Create database and tables

```
CREATE DATABASE SupplyChainDB;
USE SupplyChainDB;

CREATE TABLE
Suppliers (
supplier_id INT
PRIMARY KEY,
supplier_name
VARCHAR(100),
city VARCHAR(50) );

CREATE TABLE
Products (
product_id INT
PRIMARY KEY,
product_name
VARCHAR(100),
category
VARCHAR(50),
price DECIMAL(10,2),
stock INT,
supplier_id INT,
FOREIGN KEY
(supplier_id)
      REFERENCES Suppliers(supplier_id)
);
```

```
CREATE TABLE
Customers (
customer_id INT
PRIMARY KEY,
customer_name
VARCHAR(100),
city VARCHAR(50) );

CREATE TABLE
Orders (     order_id
INT PRIMARY KEY,
customer_id INT,
order_date DATE,
status
VARCHAR(30),
FOREIGN KEY
(customer_id)
      REFERENCES Customers(customer_id)
);

CREATE TABLE
OrderDetails (     order_id
INT,     product_id INT,
quantity INT,     unit_price
DECIMAL(10,2),
PRIMARY KEY (order_id,
product_id),
   FOREIGN KEY (order_id)
      REFERENCES Orders(order_id),
   FOREIGN KEY (product_id)
      REFERENCES Products(product_id)
);

CREATE TABLE
Deliveries (
delivery_id INT
PRIMARY KEY,
order_id INT,
delivery_date DATE,
delivery_status
VARCHAR(30),
```

```
FOREIGN KEY
(order_id)
      REFERENCES Orders(order_id)
);
```

5.2 Insert sample data

```
INSERT INTO Suppliers VALUES
(1, 'ABC Suppliers', 'Chennai'),
(2, 'Global Traders', 'Coimbatore'),
(3, 'Prime Distributors', 'Tiruppur');

INSERT INTO Products VALUES
(101, 'Laptop', 'Electronics', 50000, 20, 1),
(102, 'Mouse', 'Accessories', 500, 100, 2),
(103, 'Keyboard', 'Accessories', 1000, 5, 2),
(104, 'Monitor', 'Electronics', 12000, 15, 3);

INSERT INTO Customers VALUES
(1, 'Arun', 'Chennai'),
(2, 'Bala', 'Coimbatore'),
(3, 'Kumar', 'Tiruppur');
INSERT INTO Orders VALUES
(1001, 1, '2026-10-01', 'Delivered'),
(1002, 2, '2026-10-05', 'Shipped'),
(1003, 3, '2026-10-10', 'Processing');

INSERT INTO OrderDetails VALUES
(1001, 101, 2, 50000),
(1001, 102, 5, 500),
(1002, 103, 10, 1000),
(1003, 104, 3, 12000);

INSERT INTO Deliveries VALUES
(1, 1001, '2026-10-03', 'Delivered'),
(2, 1002, NULL, 'In Transit'),
(3, 1003, NULL, 'Pending');
```

6. Basic SQL Queries and Expected Output

6.1 Display all products

```
SELECT * FROM Products;
```

```
product_id  product_name  category      price      stock
supplier_id 101         Laptop        Electronics   50000.00
20     1
102           Mouse         Accessories      500.00  100     2
103           Keyboard      Accessories     1000.00    5     2
104           Monitor       Electronics    12000.00   15     3
```

6.2 Display all suppliers

```
SELECT * FROM Suppliers;
```

```
supplier_id  supplier_name
city 1           ABC Suppliers
Chennai
2              Global Traders      Coimbatore
3              Prime Distributors  Tiruppur
```

6.3 Display customer orders

```
SELECT c.customer_name, o.order_id, o.order_date, o.status
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id;
```

```
customer_name  order_id  order_date
status Arun           1001      2026-10-
01  Delivered
Bala          1002      2026-10-05  Shipped
Kumar         1003      2026-10-10  Processing
```

7. Analytics Queries and Expected Output

7.1 Total sales

```
SELECT SUM(quantity * unit_price) AS total_sales
FROM

OrderDetails;
```

Expected output:

```
total_sales
148500.00
```

Explanation: multiplies each ordered quantity by its unit price and adds all order line totals.

7.2 Sales for each order

```
SELECT order_id, SUM(quantity * unit_price) AS order_total
FROM OrderDetails
GROUP BY order_id
ORDER BY order_id;
```

```
order_id
order_total 1001
102500.00
1002      10000.00
1003      36000.00
```

7.3 Low-stock products (fewer than 10 units)

```
SELECT product_name, stock
FROM Products
WHERE stock < 10;
```

```
product_name  stock
Keyboard      5
```

7.4 Best-selling product by revenue

```
SELECT p.product_name,
      SUM(od.quantity * od.unit_price) AS revenue
FROM Products p
JOIN OrderDetails od ON p.product_id = od.product_id
GROUP BY p.product_id, p.product_name
ORDER BY revenue DESC
LIMIT 1;
```

```
product_name
revenue Laptop
100000.00
```

7.5 Sales by product category

```
SELECT p.category,
      SUM(od.quantity * od.unit_price) AS category_sales
FROM Products p
JOIN OrderDetails od ON p.product_id = od.product_id
GROUP BY p.category
ORDER BY p.category;
```

```
category       category_sales
Accessories    12500.00
```

Electronics 136000.00

7.6 Delivery status summary

```
SELECT delivery_status, COUNT(*) AS total_deliveries
FROM Deliveries
GROUP BY delivery_status
ORDER BY delivery_status;
```

```
delivery_status  total_deliveries
Delivered        1
In Transit       1
Pending          1
```

7.7 Supplier product count

```
SELECT s.supplier_name, COUNT(p.product_id) AS product_count
FROM Suppliers s
LEFT JOIN Products p ON s.supplier_id = p.supplier_id
GROUP BY s.supplier_id, s.supplier_name
ORDER BY s.supplier_id;
```

```
supplier_name
product_count ABC
Suppliers       1
Global Traders      2
Prime Distributors  1
```

7.8 Products never ordered

```
SELECT p.product_name
FROM Products p
LEFT JOIN OrderDetails od ON p.product_id = od.product_id
WHERE od.product_id IS NULL;
```

Empty result set (all sample products have been ordered).

7.9 Order and delivery details

```
SELECT o.order_id, c.customer_name, o.status AS order_status,
      d.delivery_status,
d.delivery_date FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
LEFT JOIN Deliveries d ON o.order_id = d.order_id
ORDER BY o.order_id;
```

```
order_id  customer_name  order_status  delivery_status
delivery_date 1001      Arun          Delivered     Delivered
2026-10-03
1002     Bala           Shipped      In Transit       NULL
1003     Kumar          Processing   Pending          NULL
```

8. Optional Improvement: Inventory Update

The following is an optional example of reducing stock after a sale. In a real application, validate available stock and update the order and inventory within a transaction.

```
START TRANSACTION;

UPDATE Products
SET stock = stock - 2
WHERE product_id = 101 AND stock >= 2;

SELECT product_name, stock
FROM Products
WHERE product_id = 101;

COMMIT;
```

With the sample data, Laptop stock changes from 20 to 18 if the update succeeds. Do not run the same sample update repeatedly unless you intend to reduce stock each time.

9. How to Run the Project

- Install MySQL Server and MySQL Workbench, or use an existing MySQL environment.
- Open a new SQL query tab.
- Run the database and table creation statements first.
- Run the INSERT statements to load the sample data.
- Execute the basic queries to inspect records.
- Execute the analytics queries and compare the results with the expected outputs.
- For a fresh rerun, remove the old database first only if you no longer need its data; otherwise use a new databasename.

10. Advantages

- Stores supply-chain data in related tables.
- Uses foreign keys to maintain valid relationships.
- Helps identify low-stock products.

- Summarizes sales by order and product category.
- Tracks delivery status and supplier product counts.
- Can be extended with a web interface, dashboards, triggers, and automated stock alerts.

11. Limitations and Future Enhancements

This academic version uses sample data and SQL reports. It does not include user login, a graphical user interface, automated purchase orders, payment processing, or a live logistics integration. Future improvements can add database views, stored procedures, triggers for stock management, date-based sales reports, role-based access, and a dashboard built with Python, Java, or a web framework.

12. Conclusion

The Supply Chain Management System with Analytics demonstrates how a relational database can support the main activities of a supply chain. The tables organize suppliers, products, customers, orders, and deliveries. The analytics queries produce total sales, order totals, category revenue, low-stock alerts, and delivery summaries. This project is suitable as a beginner-level DBMS mini project and can be expanded into a full application.